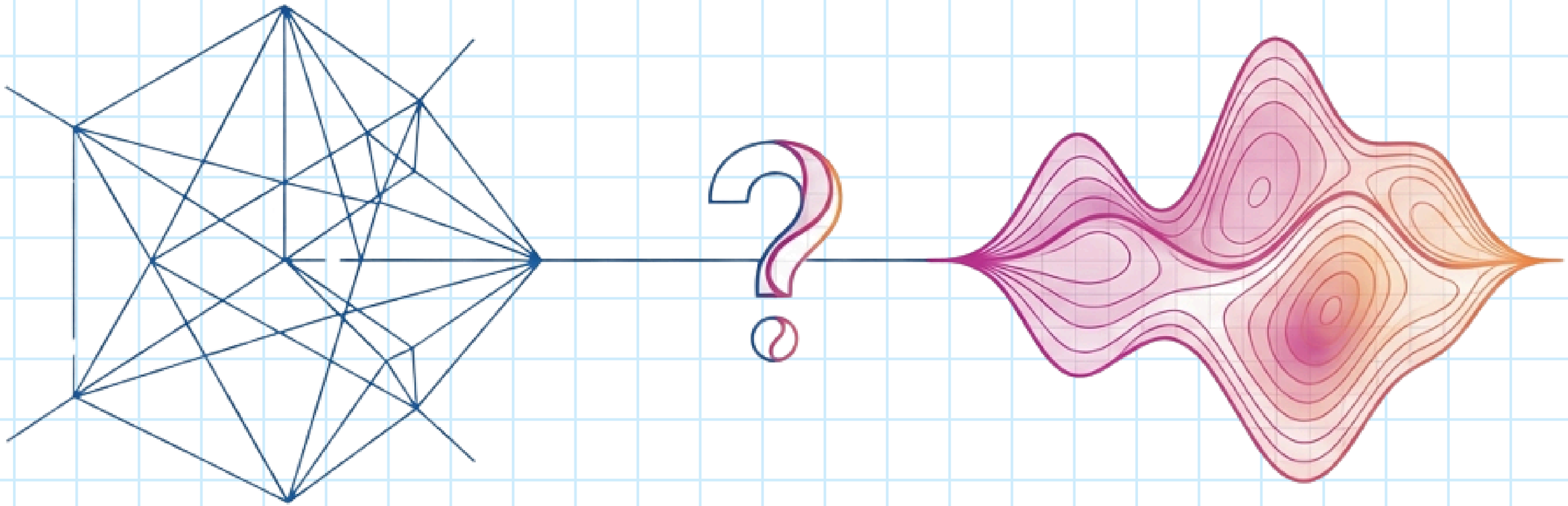
# From Linear Scores to a Single Hidden Layer

A Mathematical Study of Simple Learning Systems



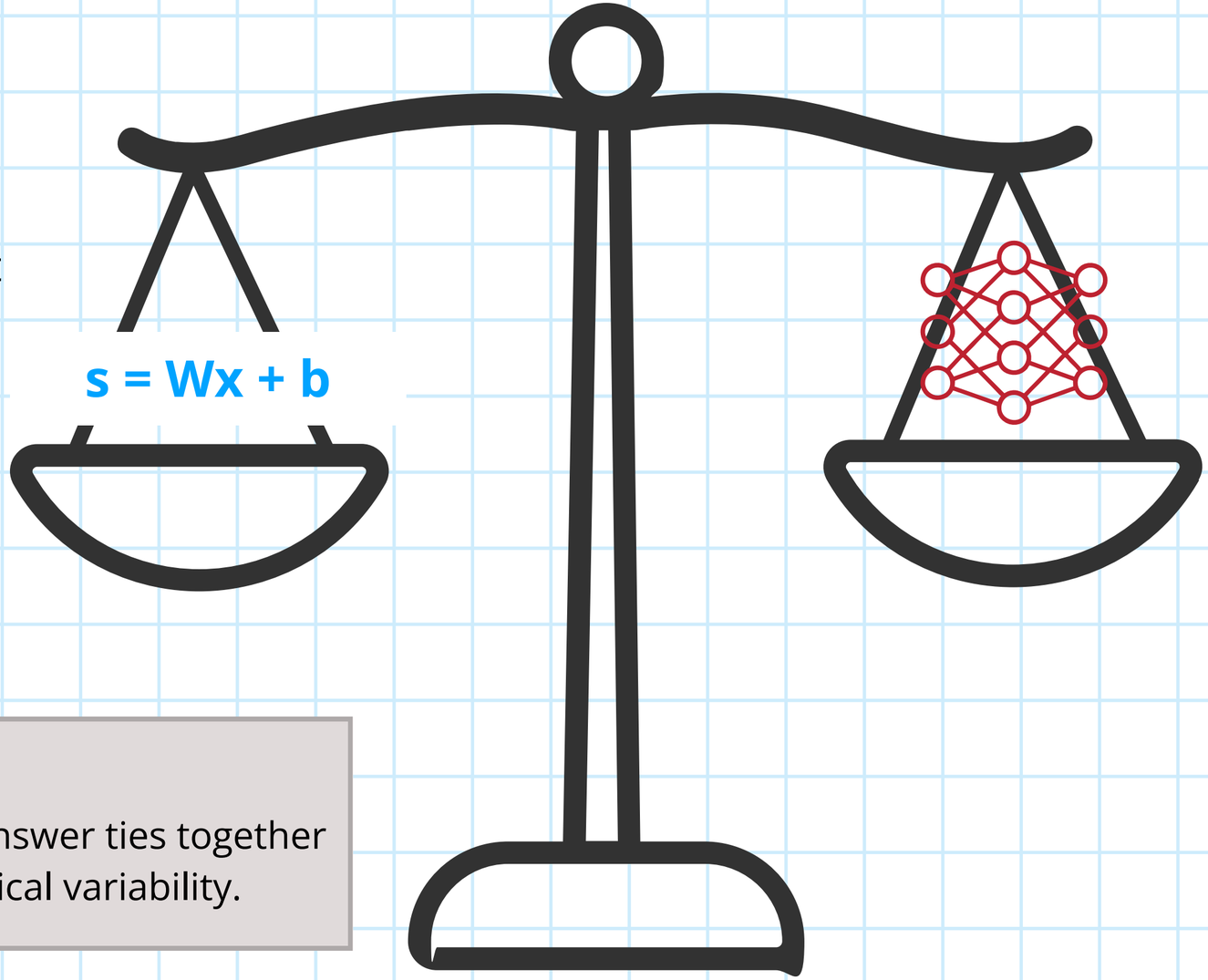
**Avaz, Kazim, Raul, and Emil**
**NAIC - AI Academy**

**Baku, Azerbaijan**
**04.04.2026**

# The Illusion of Default Complexity

**Do we always need deep learning?**

A prevailing assumption in modern AI is that more parameters automatically yield better results.



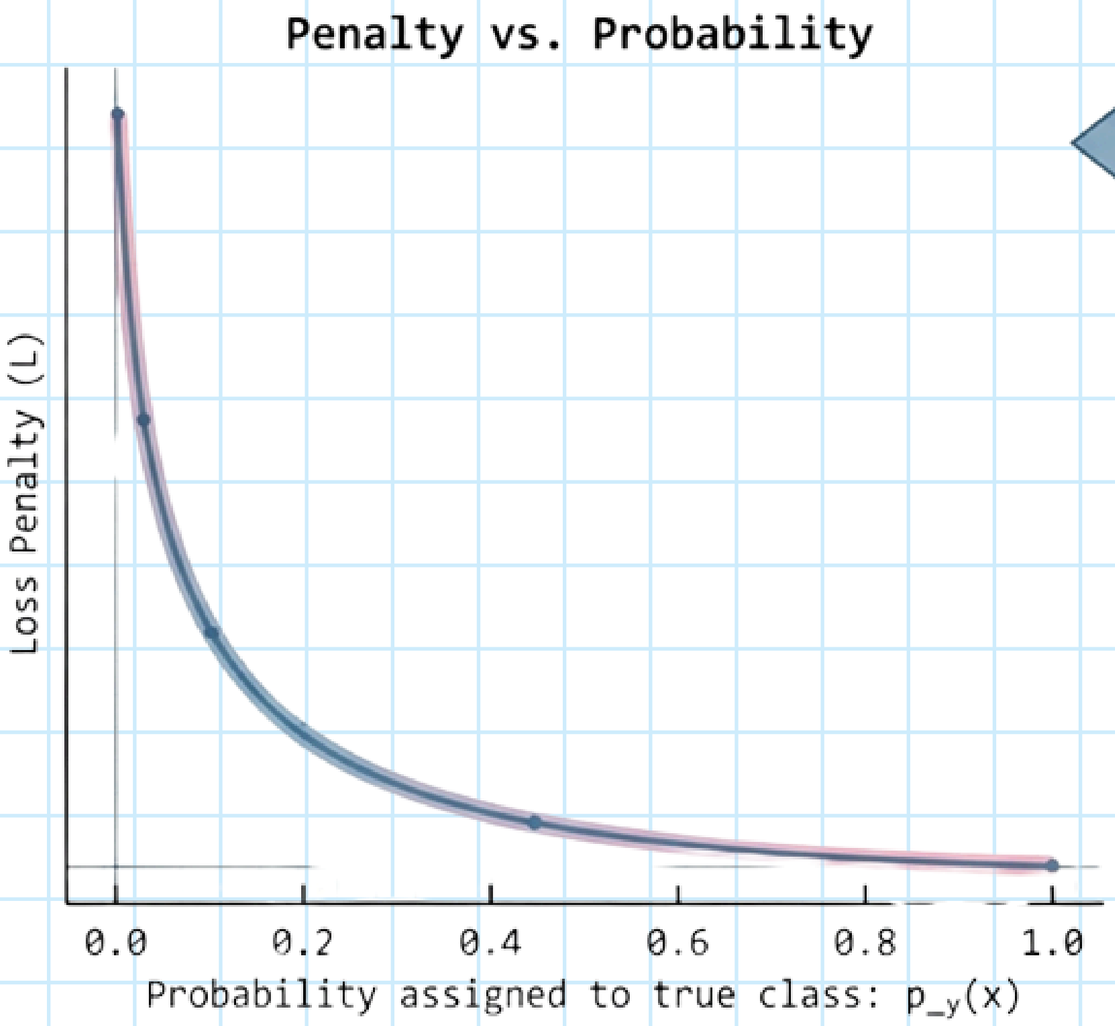

**The Core Hypothesis**

Bigger models are not automatically better. The answer ties together geometry, the chain rule, optimization, and statistical variability.

# The Contenders

| | Softmax Regression | One-Hidden-Layer NN |
|---|---|---|
| **Equations** | $s = Wx + b$ | $h = \tanh(W_1x + b_1)$<br>$s = W_2h + b_2$ |
| **Core Mechanism** | Directly scores the input space via affine transformation. | The tanh function bends and re-encodes the input space before the final linear layer scores it. |
| **Boundary Shape** | Convex polytopes (rigid hyperplanes) | Non-linear contours (warped feature geometry) |
| **Complexity** | Minimal. No hidden parameters. | Introduces hidden dimensionality (default w=32). |

# Cross-Entropy as Negative Log-Likelihood




**The Objective**

$$L(\mathbf{x}, y) = -\log p_y(\mathbf{x})$$

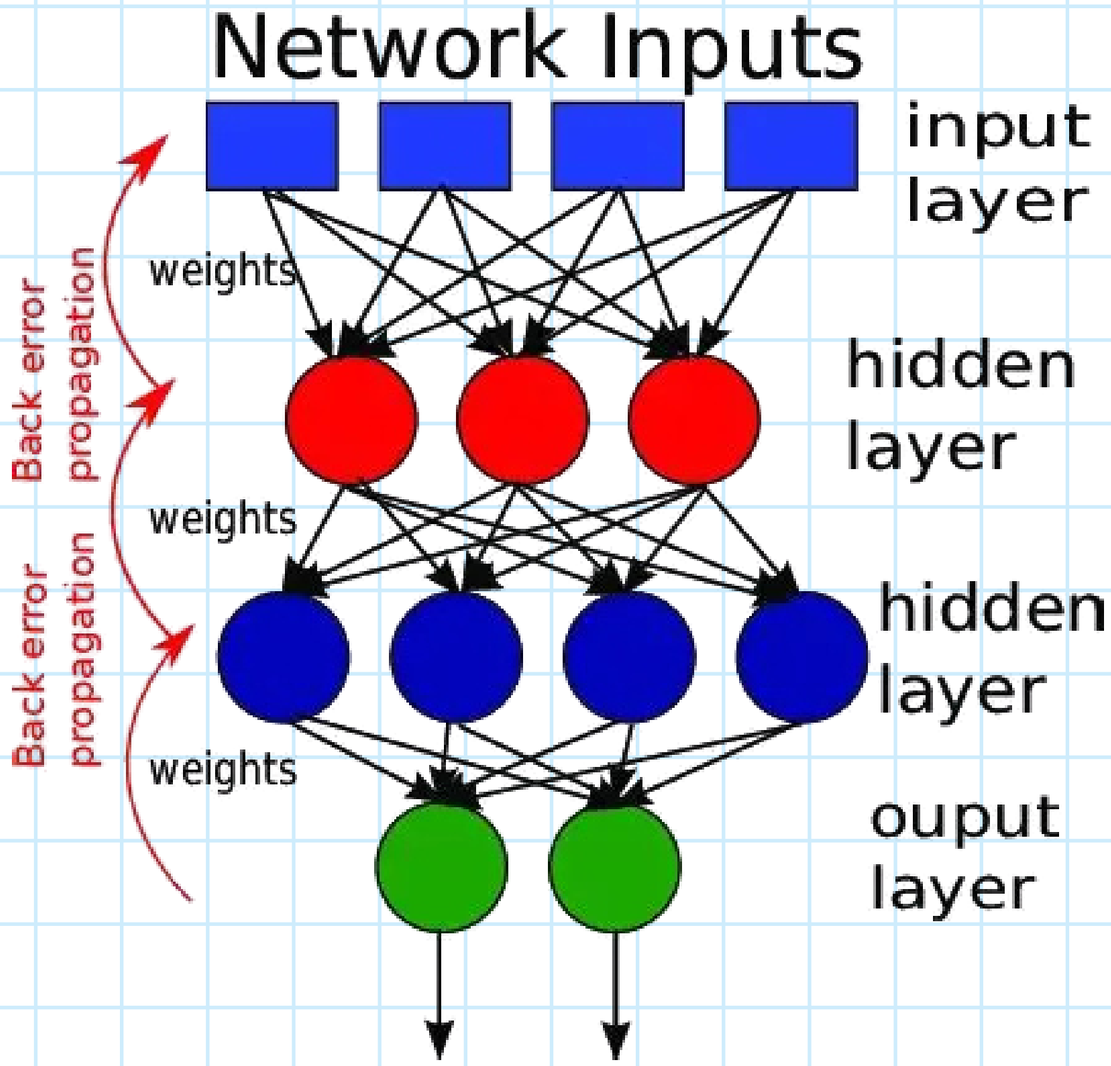
**The Mechanism**

1. The model outputs a categorical distribution over k classes.
2. The probability it assigns to the true observed label y is exactly $p_y(x)$.
3. Therefore, the log-likelihood of that observation is $\log p_y(x)$.

**The Verdict**

Averaging this penalty over n independent examples makes minimizing cross-entropy mathematically identical to maximizing likelihood.

# Backpropogation



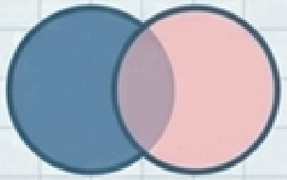

$$dZ^{[L]} = A^{[L]} - Y$$

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} (A^{[l-1]})^T, \quad l = L, L-1, \ldots, 1$$

$$db^{[l]} = \frac{1}{m} \sum dZ^{[l]}$$

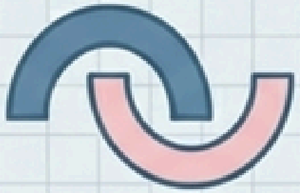
$$dZ^{[l-1]} = (W^{[l]})^T dZ^{[l]} \circ g'^{[l-1]}(Z^{[l-1]}), \quad l = L, L-1, \ldots, 2$$

# Experiments



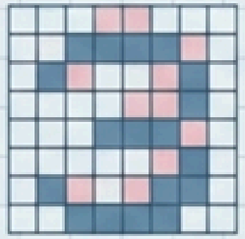
## Arena 1: Linear Gaussian

- Two synthetically generated isotropic blobs.
- Hypothesis: Approximately linearly separable.



## Arena 2: Interlocking Moons

- 2D configuration where class regions heavily interleave.
- Hypothesis: Requires a nonlinear decision boundary.

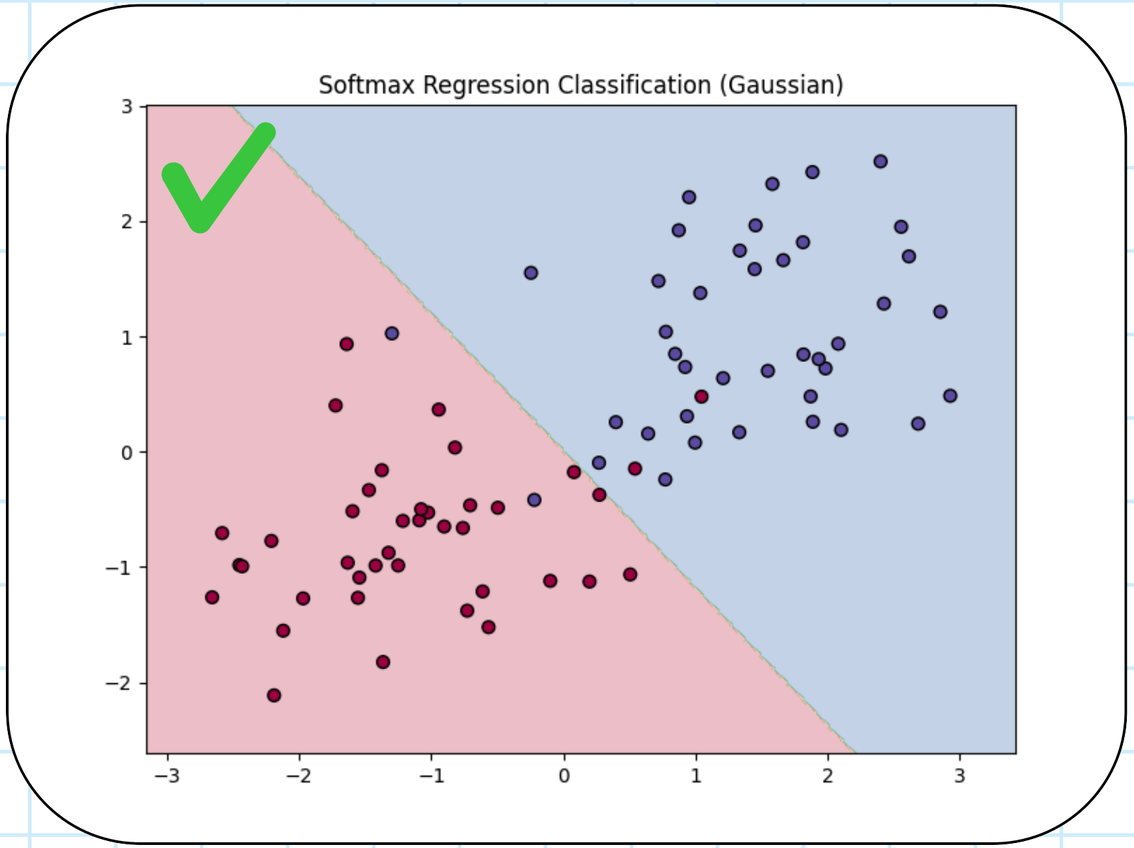


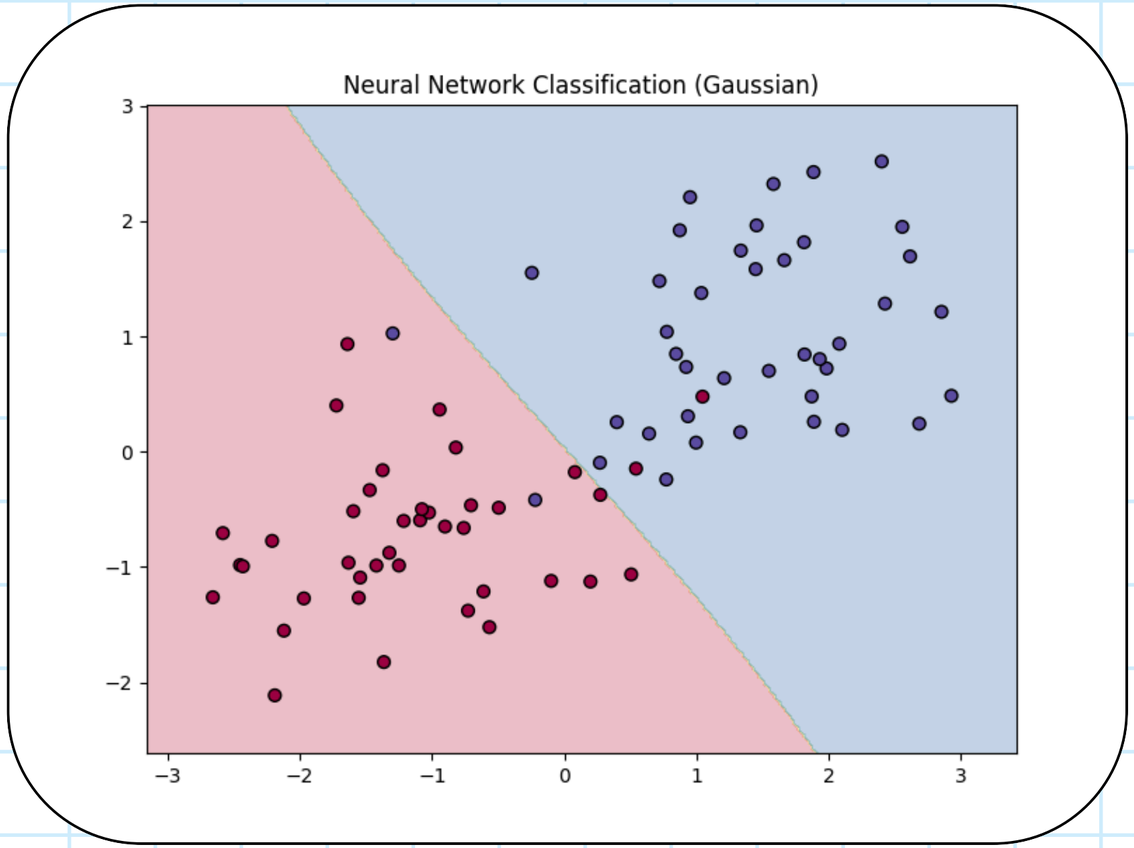
## Arena 3: Digits Benchmark

- 8x8 grayscale digit images, flattened to 64-dimensional vectors (k=10).
- Protocol: Strict train/val/test split. Five independent random seeds.

# Level 1: The Linear Gaussian Task

**Observation:** Both architectures **achieve clean separation** of the two clusters.

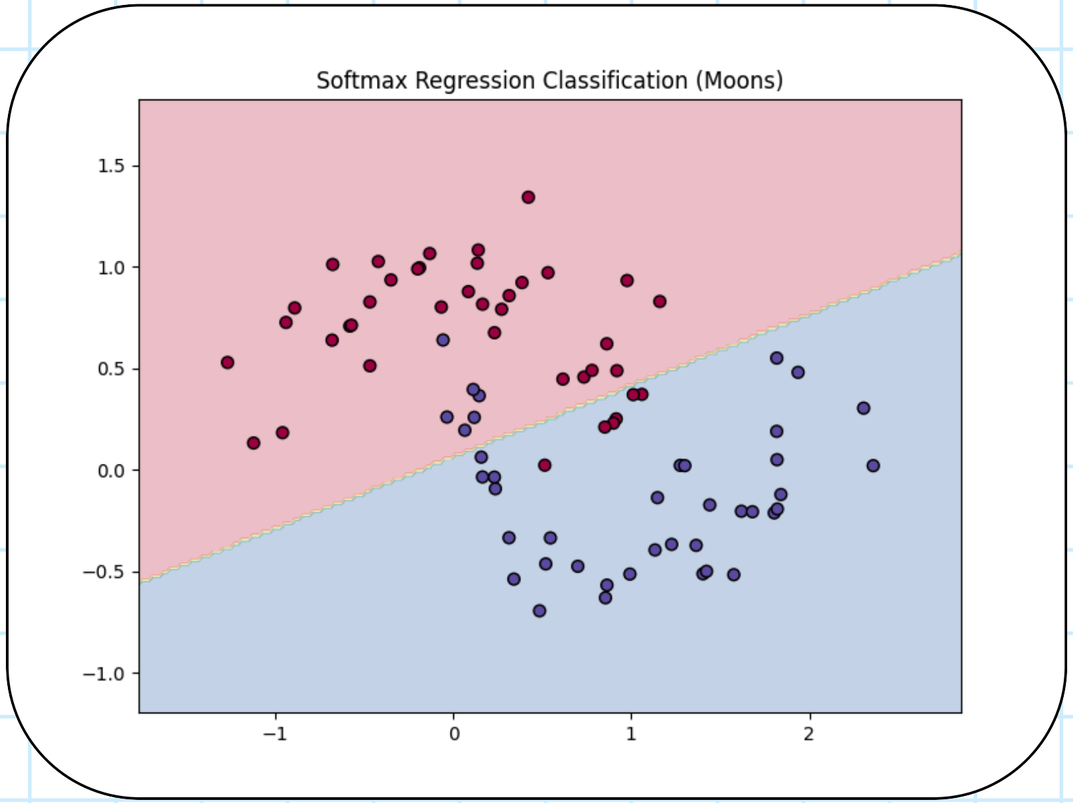


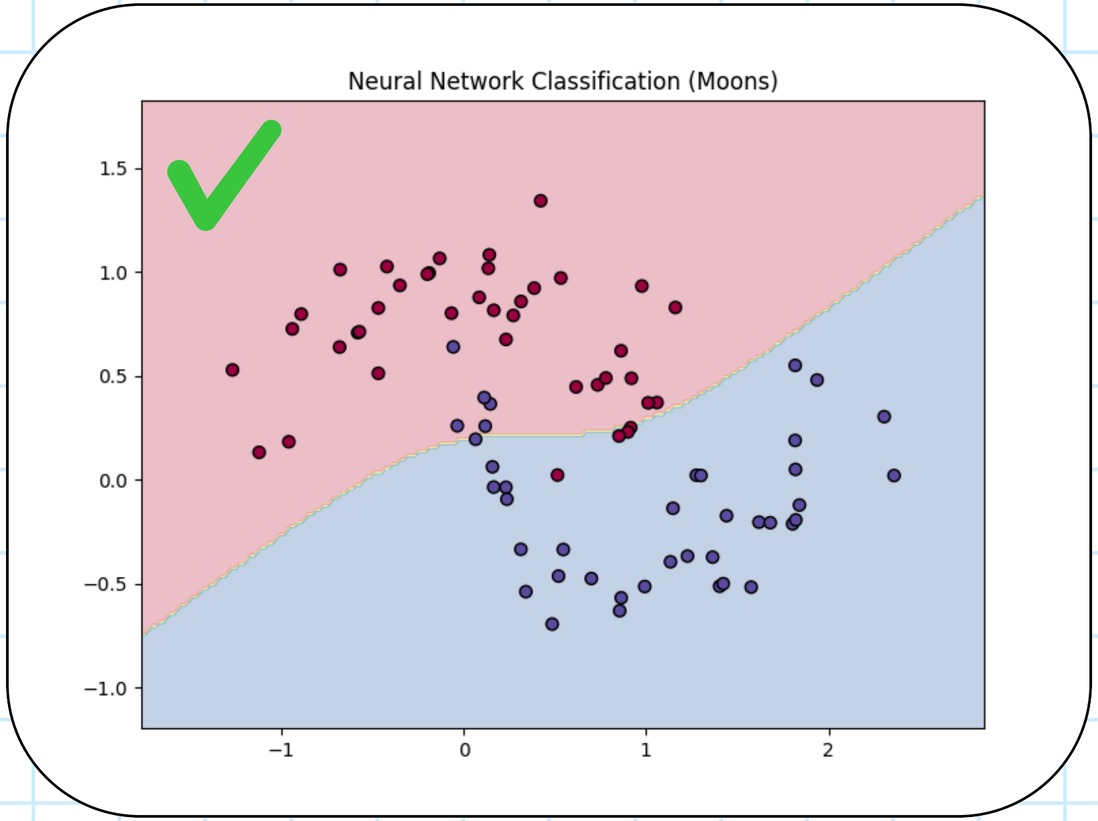




**Conclusion:**
The linear model's boundary is **sufficient**. The neural network's slightly curved boundary is merely an artifact of its flexibility and offers **no geometric advantage**. Complexity is unnecessary here.

# Level 2: The Interlocking Moons Task

**Observation:** Straight lines can't separate curved data. **Softmax** fails because it is too rigid; the **Neural Network** wins because it is flexible enough to follow the curves.

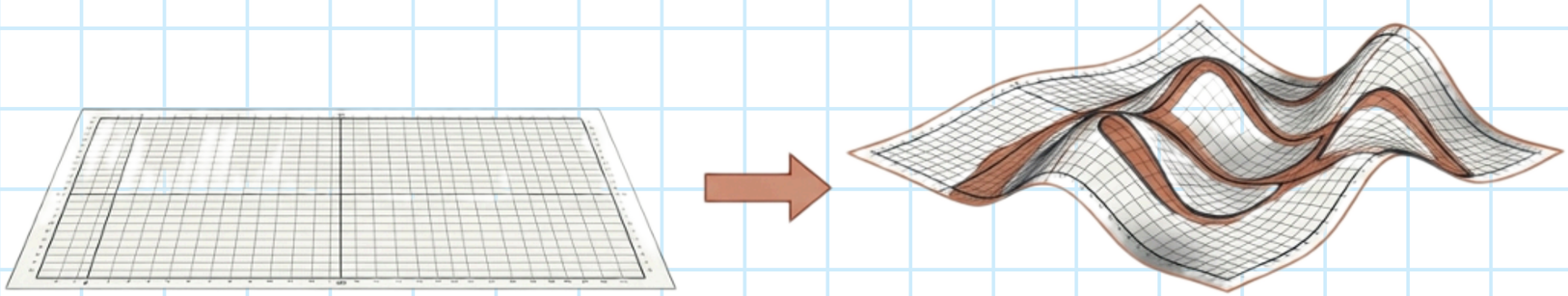


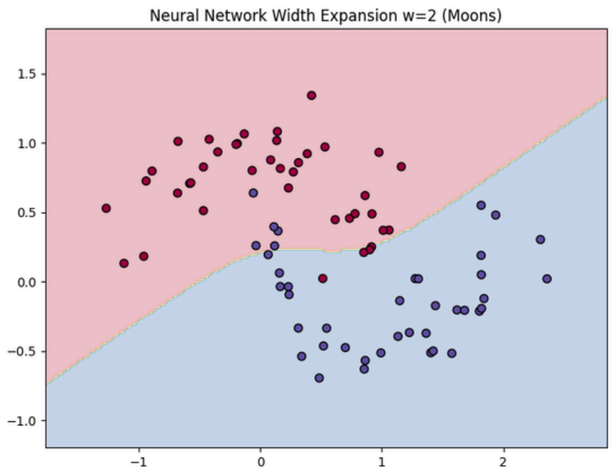



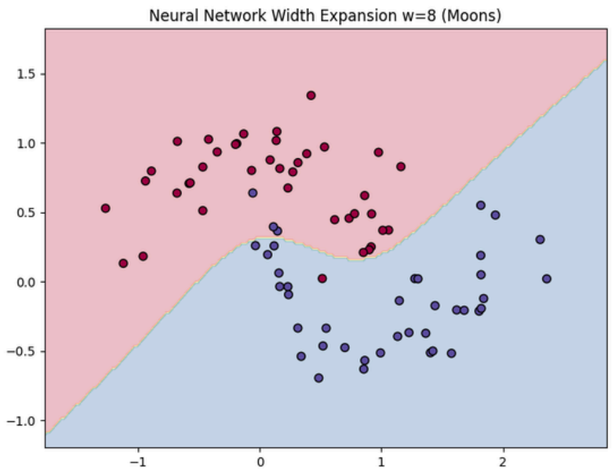

**Conclusion:**
Structural Failure. Straight lines cannot follow curves. Hidden layers are necessary to handle non-linear patterns.

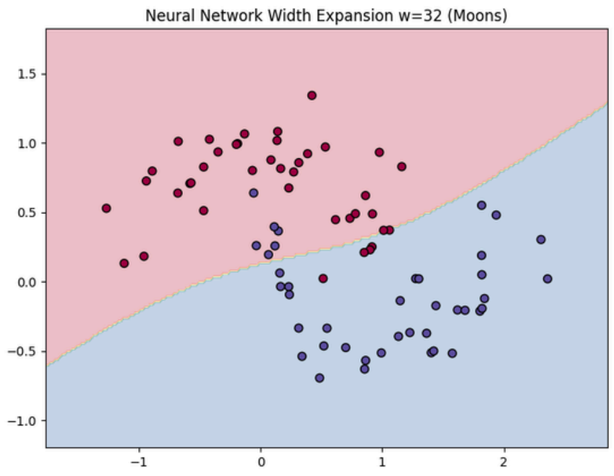
# Anatomy of a Non-Linear Fold

What does ‘capacity’ actually look like? Each hidden unit provides ones nonlinear feature fold (**tanh).**






## Width 2

Rough, low-detail separation.

## Width 8

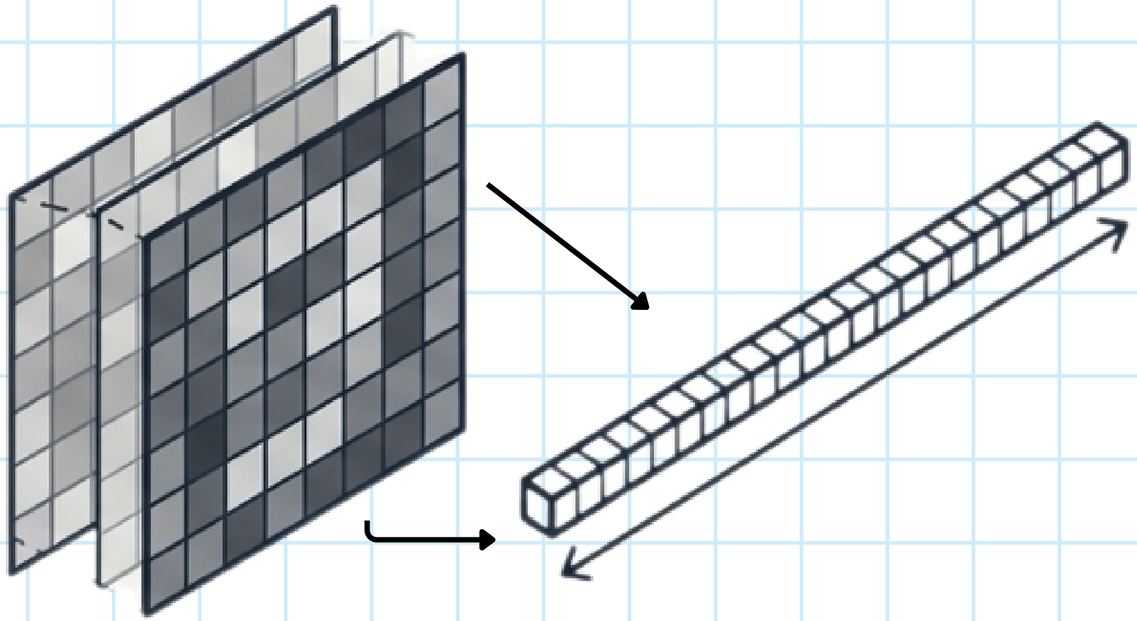
Smooth, well-balanced boundary capturing the shape.

## Width 32

Complex, high-precision boundary fitting fine details.

# Level 3: The 64-Dimensional Digits Benchmark

Evaluated over **five** independent random initializations to ensure robust, seed-agnostic statistics.



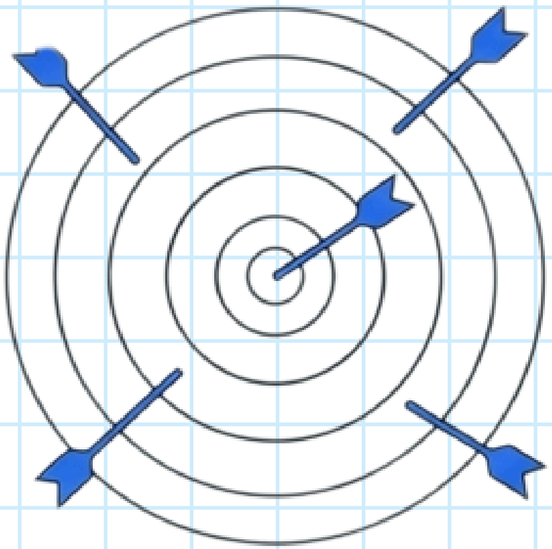
8x8 Input Grid 64-Dim Vector

| Metric | Test Accuracy |
|---|---|
| Softmax Regression | 94.73% ± 0.38% |
| Neural Network | 95.22% ± 0.38% |

**Insight:** The accuracy gap is a **mere 0.49** percentage points, and 95% confidence intervals overlap. In terms of decision quality, the two architectures are practically tied.

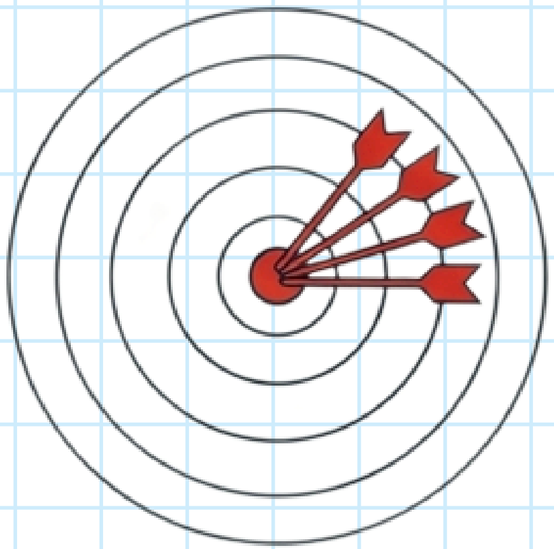
# Accuracy vs. Cross-Entropy

**Target A: Linear**



Linear Cross-Entopy Loss: 0.2315

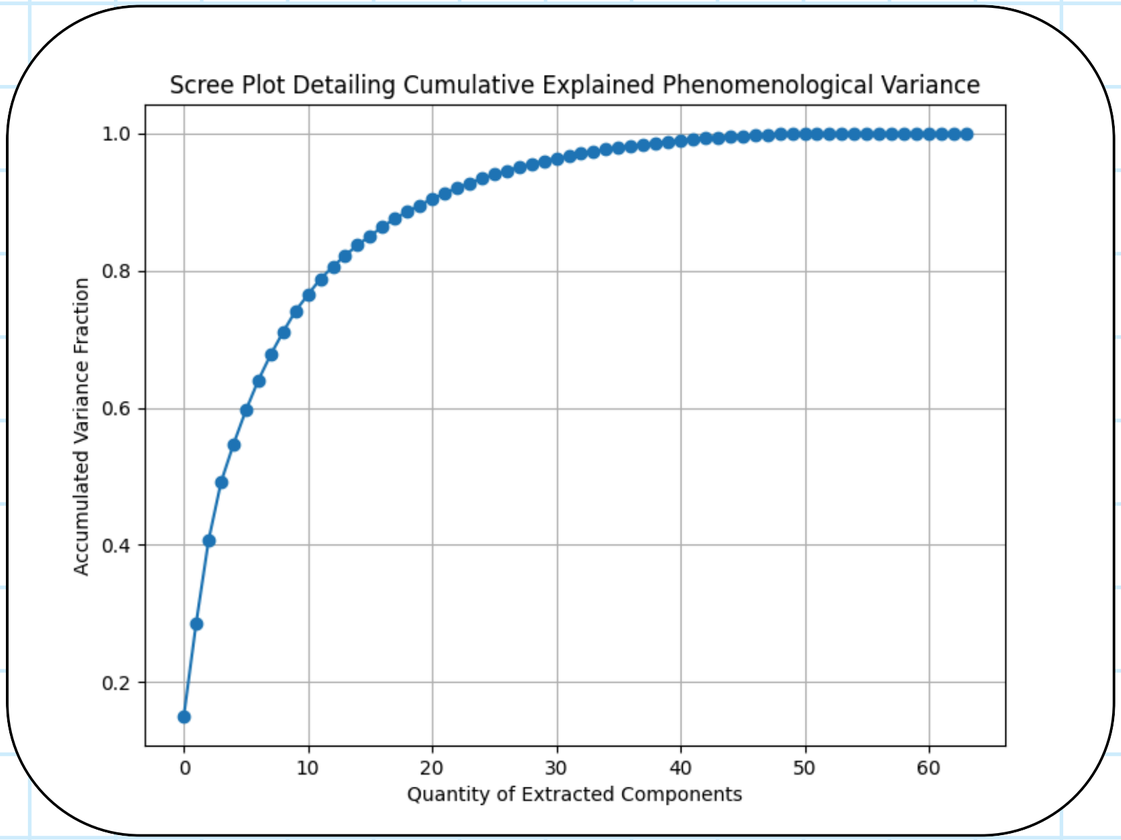
**Target B: Neural Net**



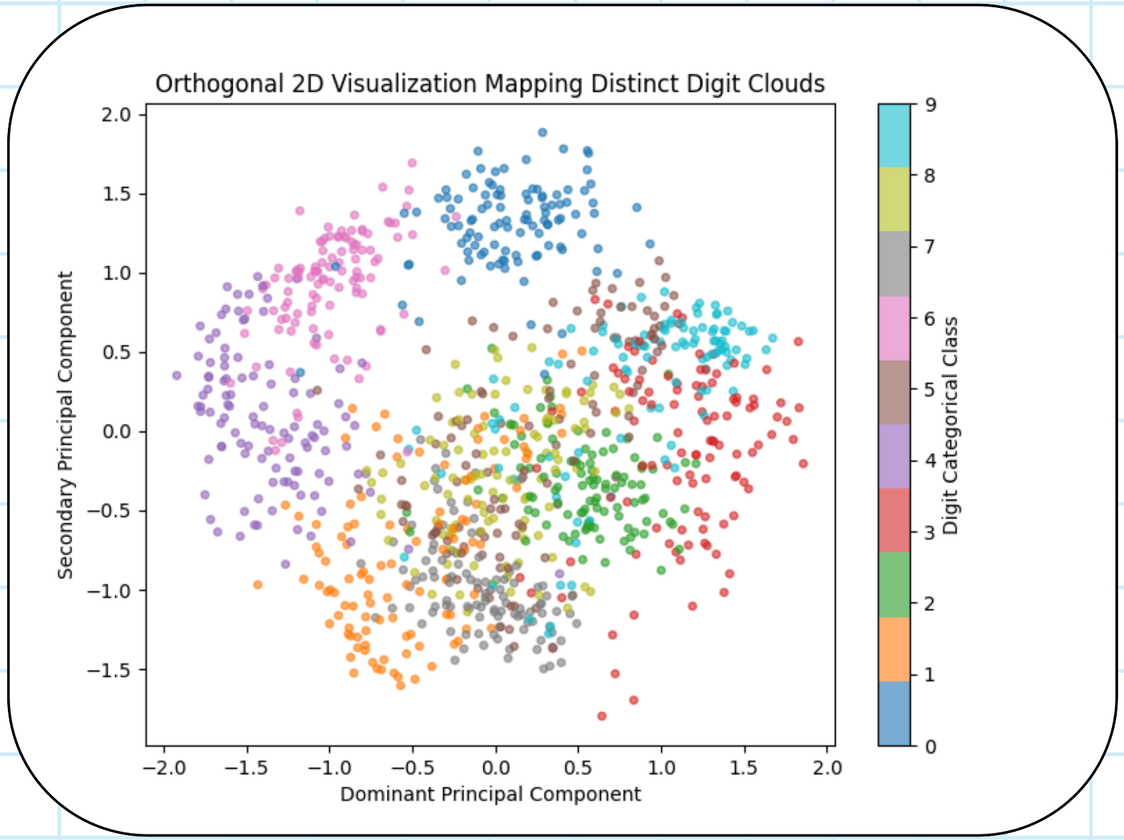
Neural Net Cross-Entropy Loss: 0.1765

**Insight:** A 23.8% reduction in average loss. While both models make similar discrete predictions, the nonlinear model is substantially better calibrated. It assigns much higher probability to the correct class; crucial for medical, autonomous, or cascading systems where confidence matters.

# Track A: PCA/SVD and input geometry



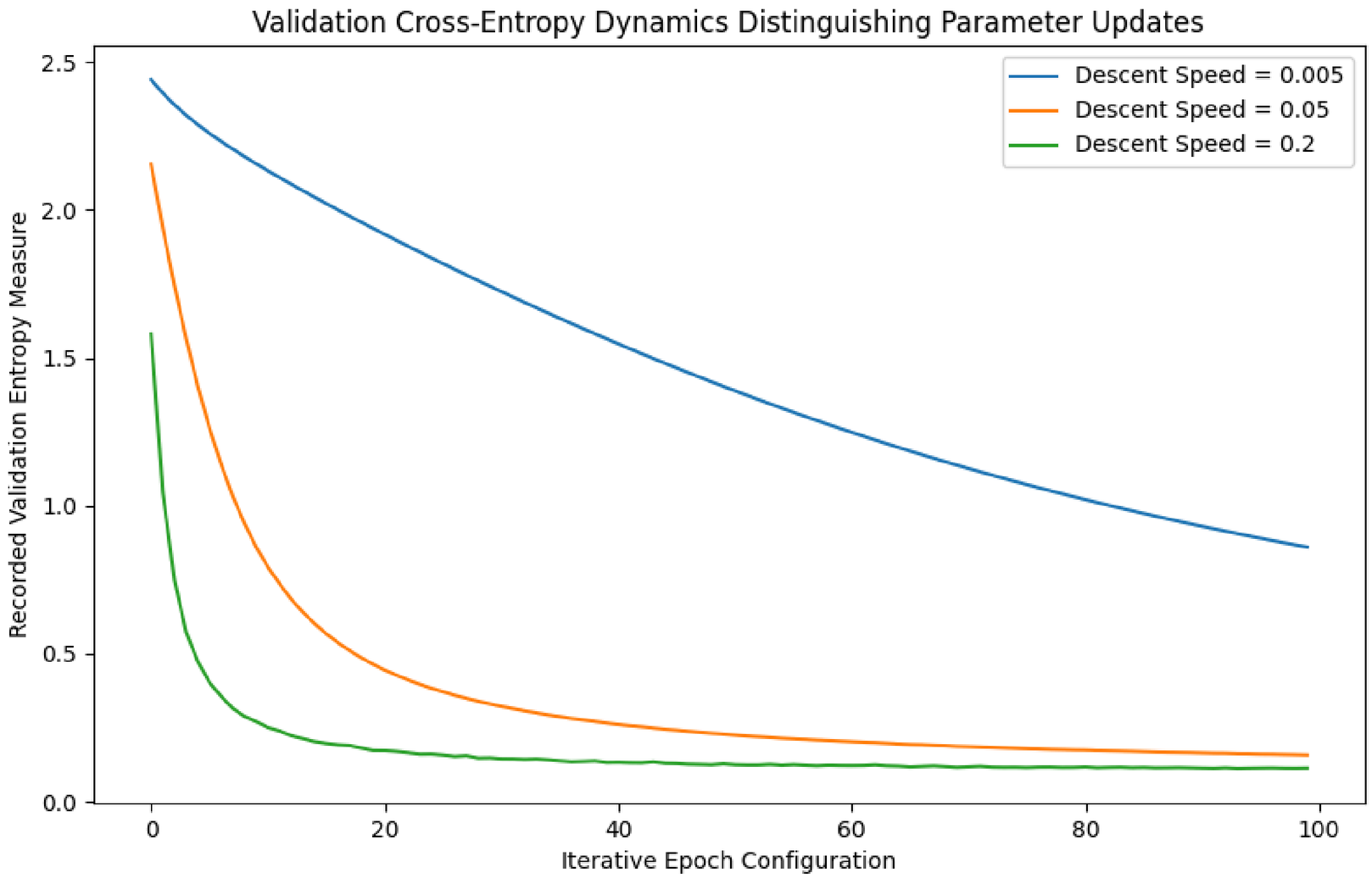

**Variance Curve:** The plot climbs rapidly, suggesting we don't need all 64 dimension.




**2D View:** Ortogonal projection reveals class overlap. Two dimensions are insufficient for classification.

**Method:** PCA analysis via SVD; $X = U\Sigma V^T$ on mean-centered features.

# Learning Rate



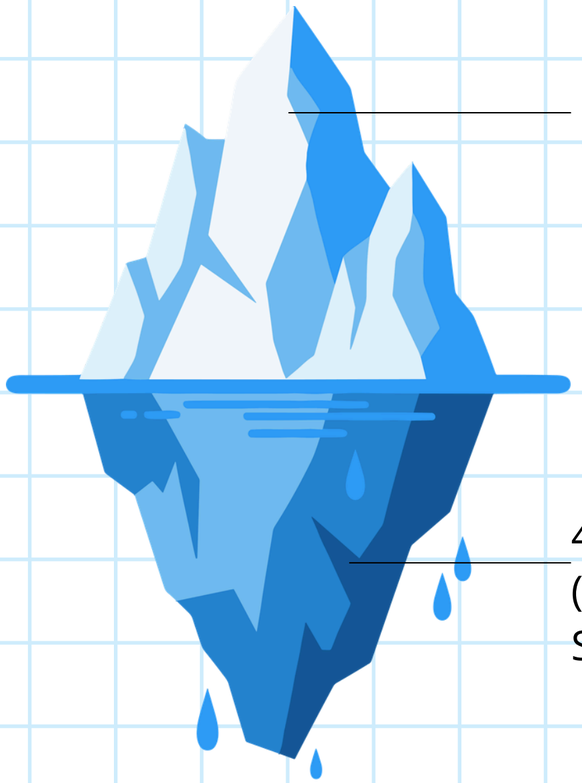

**Descent Speed = 0.2:** Rapid convergence, stabilizing near 0.1126.

**Descent Speed = 0.05:** Smooth, steady descent, settling near 0.1571.

**Descent Speed = 0.005:** Flatlines at 0.8602, essentially stuck at initialization.

**Test**: Evaluate descent speeds over 100 epochs.
**Observation**: Validation loss is highly sensitive to step size.
**Conclusion**: A good architecture fails if the step size is misaligned.

# The Low-Rank Linear Subspace




| **Retraining the Linear model on compressed inputs:** | |
|---|---|
| m=10 components | : 89.95% Accuracy |
| m=20 components | : 93.21% Accuracy (Loss: 0.2600) |
| m=40 components | : 94.57% Accuracy (Loss: 0.2316) |
| m=64 (Full baseline) | : 94.73% Accuracy (Loss: 0.2315) |

**Conclusion:**
We don't need all 64 dimensions to get high accuracy. A smaller **40-dimensional "slice"** contains almost all the useful info. Because the data lives in this flat space, a simple linear model works perfectly.

# Synthesis

| | Intrinsic Geometry | Linear Result | NN Result |
|---|---|---|---|
| Gaussian Baseline | Linearly Separable. | Clean Separation (Optimal). | Clean Separation (Unnecessary complexity). |
| Interlocking Moons | Curved / Entangled. | Structural Failure (Rigid bisector) | Smooth Contour (Geometrically essential) |
| Digits Benchmark | Low-rank linear subspace (64D → 40D). | 94.73% Acc (Highly competitive) | 95.22% Acc (Superior calibration, 23.8% loss drop) |

# The Blueprint

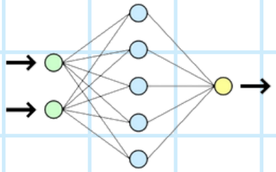
**Bigger is not inherently better.** Linear structure handles vast amounts of high-dimensional data naturally if the intrinsic rank is low.

**Understand the shape of your data before you build the engine.** Geometry dictates architecture.

**Accuracy is a blunt instrument.** Look at cross-entropy to measure true model calibration and confidence.

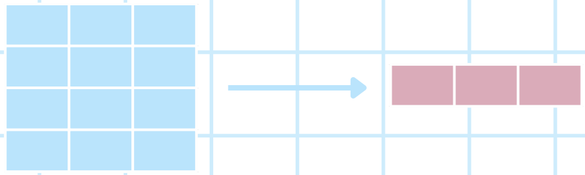
# Limitations

## Shallow Architecture



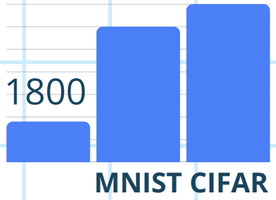
a single **tanh** layer.
DNN with ReLU behaves differently.

## No Spatial Structure



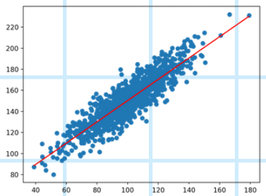
8x8 to 64.
**Ignores pixel adjacencies** that CNN exploit.

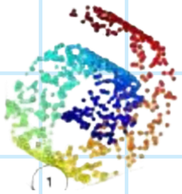
## Small Dataset




-**1800** samples.
Full **MNIST/ CIFAR** may perform better.

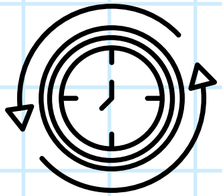
## Linear Variance Only





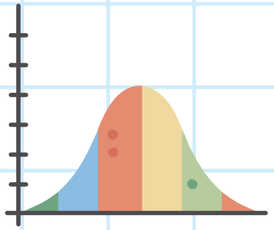
PCA captures linear structure.
**UMAP/ t-SNE** may tell richer story.

## Fixed Optimization



Constant-rate **SDG** used.
Adaptive O. may alter learning-rate sensitivity.

## Statistical Sample Size



Only **five** random seeds.
More would tighten the estimates

**Thank You!**

Any Questions?